

RESEARCH PAPER INTELLIGENCE ENGINE

Comprehensive Technology Stack & System Architecture

Document Metadata

Project Name:	Research Paper Intelligence Engine (VT)
Architecture Pattern:	Agentic RAG (Retrieval-Augmented Generation) & Multi-Stage Workflow
Primary Languages:	Python 3.10+, PowerShell (Automation Scripting)
Orchestration Framework:	LangGraph (StateGraph DAG Engine)
Vector DB & Search:	FAISS (Facebook AI Similarity Search) + Sentence-Transformers
UI Framework:	Streamlit Web Engine (Custom Animated Glassmorphism CSS)
Report Generation Date:	August 23, 2026 - 12:51:52

Executive Architecture Summary

The Research Paper Intelligence Engine is an autonomous, production-grade AI system designed to discover, process, rank, analyze, and summarize complex academic research papers. Built on a modern modular Python stack, it combines state-of-the-art Dense Vector Retrieval (FAISS), Sentence Embeddings (Sentence-Transformers), Local Transformer Models (HuggingFace/PyTorch), and a 6-node LangGraph Agentic Orchestrator to execute complex multi-paper literature reviews without manual oversight.

Section 1: User Interface & Presentation Layer

Primary Files: `app.py`

The presentation layer provides a web dashboard featuring multi-tab navigation, custom progress trackers, interactive QA interfaces, and dynamic visualization widgets.

Component	Technology / Library	Description & Role
Framework	Streamlit >= 1.30.0	Python web framework for data & AI apps
Styling & Theme	Vanilla CSS / HTML5	Glassmorphism UI, custom dark-mode gradients, smooth animated tab transitions
Interactivity	Streamlit Session State	Stateful management for uploaded files, active tabs, RAG chat history, and workflows
Media Handling	PyMuPDF / Streamlit PDF Viewer	Embedded PDF document previews and interactive chat UI

Section 2: Agentic AI Workflow & Orchestration Engine

Primary Files: `src/agent.py`, `src/analyzer.py`, `run_agent_test.py`

The Agentic layer automates research planning, arXiv paper retrieval, semantic relevance ranking, FAISS ingestion, structured analytical extraction, comparative matrix creation, and literature review synthesis.

Component	Technology / Library	Description & Role
Orchestrator	LangGraph >= 0.0.20	StateGraph directed acyclic graph (DAG) engine for autonomous 6-stage workflow
State Management	TypedDict (ResearchState)	Strongly typed state container tracking topic, questions, candidates, rankings, and
Schema Validation	Pydantic >= 2.0.0	BaseModel schema enforcement for structured extraction of problem statements,
Paper Discovery	arxiv >= 2.1.0 API Client	Automated fetching and local PDF downloading from arXiv open-access database

Section 3: Document Ingestion & Text Processing

Primary Files: `src/pdf_processor.py`, `src/chunking.py`

Converts binary research PDF documents into structured text blocks, preserving page metadata, titles, abstracts, and key section headings for downstream indexing.

Component	Technology / Library	Description & Role
PDF Parser	PyMuPDF (fitz) >= 1.23.0	High-speed text, layout, and metadata extraction from raw PDF files
Text Splitter	LangChain RecursiveCharacterText	Context-aware chunking with customizable chunk sizes (1000 chars) and overlap
Sanitization	Python re (Regex Engine)	Unicode sanitization, noise removal, reference stripping, and header extraction

Section 4: Embedding Generation & Tensor Computing

Primary Files: `src/embeddings.py`

Transforms text chunks and natural language queries into normalized dense vector embeddings for semantic vector indexing.

Component	Technology / Library	Description & Role
Embedding Library	sentence-transformers >= 2.2.2	Dense vector embedding generation using Sentence-BERT architectures
Default Models	all-MiniLM-L6-v2 / BAAI/bge-small-en-v1.5	384-dimensional dense vectors fine-tuned for semantic text similarity
Tensor Backend	PyTorch >= 2.0.0 / NumPy >= 1.24.0	CPU tensor computations with automated HuggingFace model cache directory management

Section 5: Vector Storage & Retrieval Engine

Primary Files: `src/vector_db.py`, `src/retriever.py`, `src/ranker.py`

Enables millisecond-level similarity search over indexed research paper chunks to retrieve relevant contextual snippets for RAG QA.

Component	Technology / Library	Description & Role
Vector Store	FAISS CPU (faiss-cpu >= 1.7.4)	Facebook AI Similarity Search engine using IndexFlatL2 dense vector index
Ranking Engine	SemanticRanker (Cosine Similarity)	Reranking candidate papers against research topics using sentence embeddings
Persistence	FAISS binary index + Pickle	On-disk index persistence and fast warm-start vector database loading

Section 6: RAG Pipeline & QA Generation

Primary Files: `src/rag_pipeline.py`, `src/summarizer.py`

Synthesizes precise, grounded answers to user questions using retrieved paper chunks, preventing hallucinations through strict context constraint.

Component	Technology / Library	Description & Role
Pipeline Framework	LangChain >= 0.1.0	Retrieval-Augmented Generation chain combining FAISS retrieval with LLM inference
Transformers	HuggingFace Transformers >= 4.36	Pretrained transformer pipelines for question answering and summarization
Default Models	google/flan-t5-base / facebook/bart-l	Sequence-to-sequence text generation and abstractive summarization models
Fallback Engine	Heuristic Context Analyzer	Deterministic fallback engine ensuring answer availability even in offline / low-reso

Section 7: Citation Graph Analysis & Export Engine

Primary Files: `src/citation_graph.py`, `src/export.py`

Builds citation connectivity graphs and exports comprehensive formatted research reports in PDF format.

Component	Technology / Library	Description & Role
Citation Engine	Custom Regex & PyMuPDF	Parses paper bibliography sections, extracts inline citation keys, and builds paper
PDF Generator	fpdf2 >= 2.7.0	Programmatic PDF document generation for research summary reports, metrics e

Section 8: DevOps, Automation & Scripts

Primary Files: `config.py`, `setup_local.ps1`, `start_app.ps1`, `build_colab_notebook.py`

Provides zero-configuration local setup scripts for Windows environments and notebook builders for Google Colab.

Component	Technology / Library	Description & Role
Configuration	Python Config (<code>config.py</code>)	Centralized environment settings, log levels, model parameters, and path specific
Windows Automation	PowerShell (<code>ps1</code>)	Automated virtual environment creation, pip dependency installation, and server s
Notebook Integration	Jupyter / Google Colab	Standalone notebook runners (<code>Research_Engine_Standalone.ipynb</code>) for cloud GP

Master Dependency & Package Matrix

Package Name	Version Req.	System Functional Purpose
PyMuPDF (fitz)	>= 1.23.0	High-performance PDF document parsing and text extraction
langchain	>= 0.1.0	Core RAG framework, text splitters, and LLM chain abstractions
langchain-community	>= 0.0.20	Community integrations and extended vector store splitters
langgraph	>= 0.0.20	StateGraph engine for multi-agent stateful workflow orchestration
sentence-transformers	>= 2.2.2	Pretrained transformer embeddings for dense semantic retrieval
faiss-cpu	>= 1.7.4	Facebook AI Similarity Search engine for efficient vector search
transformers	>= 4.36.0	HuggingFace deep learning models for QA and summarization
torch	>= 2.0.0	PyTorch deep learning framework for model execution & tensor math
streamlit	>= 1.30.0	Interactive web dashboard framework
pydantic	>= 2.0.0	Data validation, state management, and schema enforcement
arxiv	>= 2.1.0	Python wrapper for arXiv API research paper retrieval
fpdf2	>= 2.7.0	PDF generation library for reporting and specifications
numpy	>= 1.24.0	Array and matrix numerical calculations
tqdm	>= 4.66.0	Console and UI progress bars for ingestion pipelines